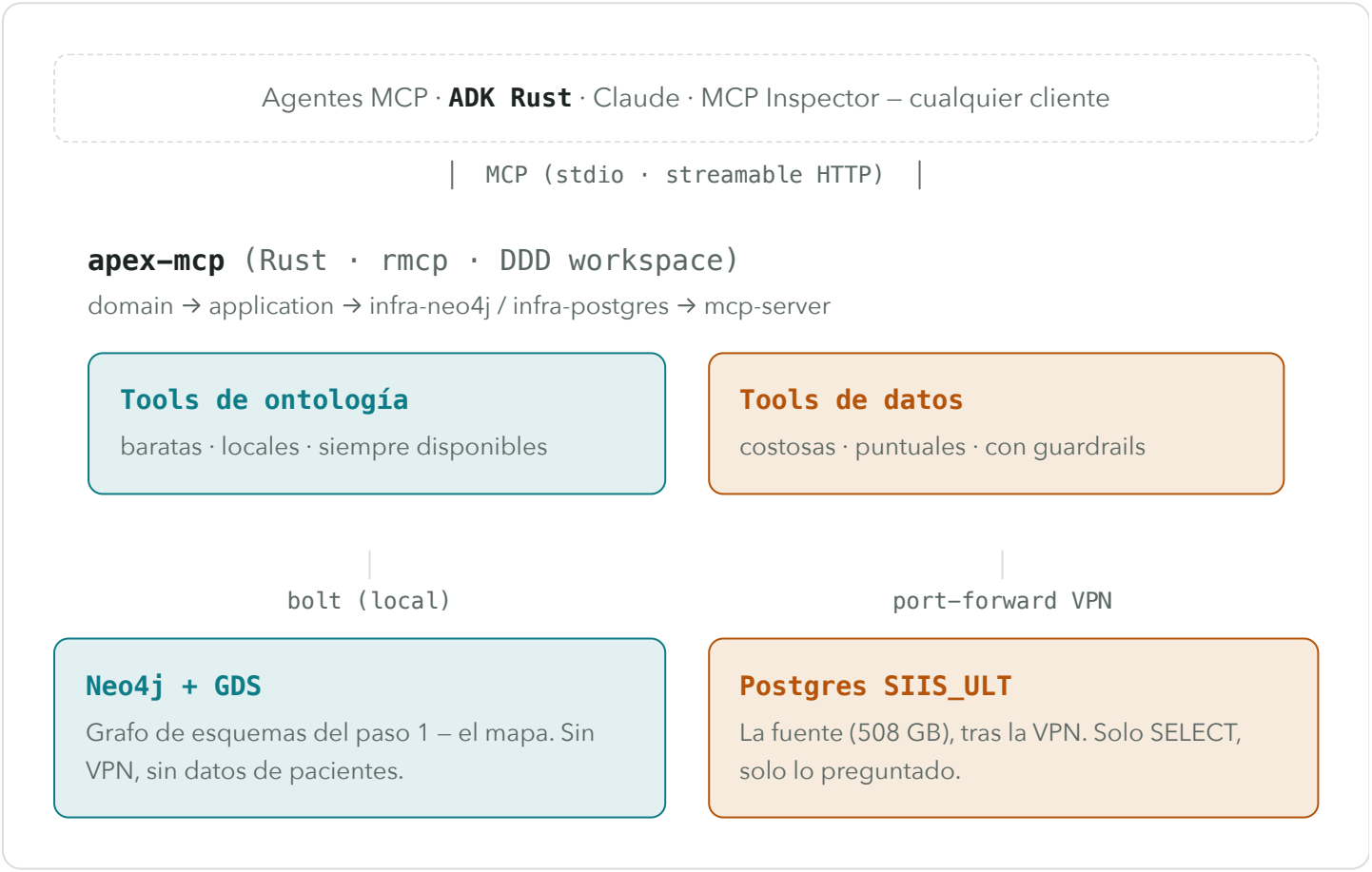
APEX DS PIPELINE · PASO 2 · PROPUESTA DE ARQUITECTURA

apex-mcp: la ontología responde primero, la fuente solo cuando hace falta

Un servidor MCP en Rust que expone el grafo de conocimiento del paso 1 (**4,542 tablas, 7,763 FKs**) a cualquier agente LLM, y ejecuta consultas puntuales sobre el Postgres de la clínica en runtime — **federación, no replicación**: nunca se copia la base completa.

ARQUITECTURA



LAS DOS FAMILIAS DE TOOLS

ONTOLOGÍA — EL GRAFO ENRUTA

`search_tables`	encuentra las ~10 tablas relevantes entre 4,542 (fulltext → v2 embeddings)
`describe_table`	columnas, tipos, null_frac, PKs/FKs, comentarios
`join_path`	camino de FKs entre dos tablas — cómo joinear sin adivinar
`table_neighborhood`	subgrafo alrededor de una entidad
`domain_overview`	esquemas + tablas top por tamaño y centralidad (GDS)

Responden en milisegundos y funcionan aunque la VPN esté caída — el grafo vive local.

DATOS — LA FUENTE RESPONDE

`aggregate_table`	agregados por plantilla (count/sum/avg + group by) — el camino feliz
`run_read_query`	SELECT libre pero vigilado: parser, read-only, timeout, LIMIT
`sample_rows`	muestra pequeña — apagado por defecto (datos clínicos)
`source_status`	¿túnel vivo? latencia — el agente sabe antes de intentar

Cada llamada queda auditada (tracing → OTEL/ClickHouse en APEX ARCH).

EJEMPLO END-TO-END

Usuario pregunta: `"¿Cómo está la cartera? ¿quiénes son los mayores deudores?"`

1	`search_tables("cartera")` → esquema **cartera** (1.1 GB) + **cuentas_detalle**, facturas… rankeadas	`~ms · local`
2	`describe_table` + `join_path` → estructura y JOIN por FK explícita	`~ms · local`
3	`aggregate_table` → `sum(saldo) group by tercero · top 20`, read-only, 15 s máx	`~s · VPN`
4	El agente responde con números reales y el linaje de dónde salieron	`total < 10 s`

El grafo redujo 4,542 tablas al puñado correcto (gratis); la fuente ejecutó *una* agregación puntual (mínimo costo). No se copió ni una tabla.

GUARDRAILS — LA FUENTE ES UNA BASE CLÍNICA EN PRODUCCIÓN

`solo lectura`
Parser acepta solo SELECT/WITH · transacción READ ONLY en Postgres

`presupuesto por consulta`
statement_timeout 15 s · LIMIT inyectado (≤ 500 filas) · 1 statement

`privacidad primero`
Agregados como camino feliz · sample_rows off por defecto

`auditoría total`
Toda consulta se registra con quién, qué y cuánto costó

ROADMAP

FASE	QUÉ ENTRA	QUÉ HABILITA
MVP	Tools de ontología (fulltext + GDS) · `aggregate_table` / `run_read_query` con guardrails · stdio	Insights de cartera (y cualquier dominio) hoy, con el grafo que ya existe
`v1.1`	Streamable HTTP + auth · auditoría estructurada	Agentes ADK Rust remotos (flecha "MCP Communication" del diagrama)
`v2`	Índice vectorial + embeddings en Neo4j (Schema RAG)	`search_tables` semántico — misma firma, mejor recall
`v2.1`	ClickHouse / R2-Parquet como backend caliente	Preguntas repetidas dejan de pagar la VPN (pasos 2–3 del diagrama DS)
`v3`	CDC con Debezium	Grafo y cachés siempre frescos (paso 4)

Diseño completo con estructura DDD, contratos de tools, pruebas (MCP Inspector, sin LLM) y riesgos: `apex-mcp/DESIGN.md` · Insumos: apex-data-profiling-pipeline.pdf · agentic_data_workflows_rust.pdf · rust-sdk oficial de MCP (rmcp) · grafo real del paso 1 ya cargado en Neo4j local.